SEMANA 3 · AULA 1 · 03/11

STEERING E SKILL

As regras da casa, o método, e a sua primeira skill de code review

Felipe Oliveira · IA Aplicada à Produtividade no Desenvolvimento

<LAB365> SENAI

A SEMANA EM UM SLIDE

A PERGUNTA

Como transformar o que eu sei em algo que o agente faz do mesmo jeito toda vez, em qualquer ferramenta?

A RESPOSTA

Três camadas de contexto: steering (regras da casa), skill (método) e spec (o que construir agora).

O DOMÍNIO

Code review com agentes, pedido da turma. Um repo de pagamentos com bugs plantados.

O JOGO

Disputa em toda aula; na quinta, o Torneio Red Team x Blue Team.

3
camadas, uma por aula

14
bugs plantados no repo

3
ferramentas, mesmo conteúdo

1
torneio na quinta

TRÊS CAMADAS, TRÊS PERGUNTAS

STEERING

Como trabalhamos neste repositório?

escopo: o projeto inteiro **quando:** sempre, em todo turno **carrega:** regras, stack, convenções

SKILL

Como se faz este tipo de tarefa?

escopo: um tipo de tarefa, em qualquer repo **quando:** sob demanda, quando a descrição casa
carrega: método, checklist, scripts

SPEC

O que estamos construindo agora?

escopo: uma feature **quando:** uma vez, até entregar **carrega:** requisitos, design, tarefas

Leitura: do mais amplo e permanente para o mais estreito e efêmero. Os três convivem na mesma sessão.

AGENDA DA AULA

19h00	Org SENAI-IA: equipes, apelidos, logos e repos	GITHUB
19h10	As três camadas e a pergunta da semana	ABERTURA
19h15	Bloco 1: steering, as regras da casa	TEORIA
19h35	Atividade 1.1: steering do pagamentos-api	PRÁTICA
19h45	Bloco 2: skill, o método; taxonomia	TEORIA
20h00	Intervalo	PAUSA
20h10	Atividade 1.2: Dissecação de 4 skills	PRÁTICA
20h30	Bloco 3: anatomia de uma skill	TEORIA
20h50	Atividade 1.3: skill v0 + Disputa 1	DISPUTA

ENTREGÁVEIS DE HOJE

Steering + skill v0

- arquivo de steering do repo
- skill code-review-<grupo>
- frontmatter válido na spec
- relatório reports/<grupo>-v0.md
- placar da Disputa 1

A ORG SENAI-IA NO GITHUB

TURMA E EQUIPES

Turma IAPD-26T3. Equipes IAPD-26T3-E01, E02...: o código não muda; o apelido vem depois do hífen.

REPOSITÓRIOS

Equipe: iapd-26t3-e03-<projeto>. Turma: iapd-26t3-materiais, uma pasta por aula.

TRABALHO INDIVIDUAL

Pasta com o seu usuário dentro do repo da equipe, branch própria, entrega por Pull Request.

PORTFÓLIO

No fim da turma os repos são arquivados e viram portfólio. Capriche no README.

AGORA, EM 10 MINUTOS

Cada equipe faz

- escolhe o apelido da equipe
- manda o logo (PNG quadrado)
- cria iapd-26t3-eNN-skills com README
- vincula o time como Admin
- dúvidas: @SENAI-IA/professores

BLOCO 1: STEERING

As regras da casa: o que o agente precisa saber em toda tarefa deste repositório

STEERING: AS REGRAS DA CASA

O QUE É

Markdown no repo, lido pelo agente sem ninguém pedir. Fatos que valem para qualquer tarefa neste projeto.

O QUE VAI

Stack, comandos de build e teste, layout de pastas, convenções e gotchas curtos.

O QUE NÃO VAI

Procedimento longo (vira skill), regra que não pode falhar (vira hook), o que o modelo já sabe.

TAMANHO

Poucas dezenas de linhas. Entra em todo turno: cada linha custa em toda mensagem.

TESTE DE PERTENCIMENTO

Vai para o steering se...

- é verdade hoje e daqui a 3 meses
- vale para qualquer tarefa no repo
- o modelo erraria sem saber
- cabe em uma ou duas linhas

ONDE O STEERING MORA

FERRAMENTA	SEMPRE CARREGADO	CONDICIONAL POR ARQUIVO	OBSERVAÇÃO
Kiro	.kiro/steering/*.md (inclusion: always)	inclusion: fileMatch + padrão	também manual e auto
Claude Code	CLAUDE.md na raiz; .claude/rules/*.md	.claude/rules com paths:	CLAUDE.md em subpasta carrega ao entrar nela
VS Code + Copilot	.github/copilot-instructions.md	.github/instructions/*.instructions.md com applyTo:	lê CLAUDE.md e AGENTS.md também
Neutro: AGENTS.md	AGENTS.md na raiz	AGENTS.md aninhado por pasta	Copilot, Codex, Cursor, Gemini CLI

Agnóstico na prática: escreva um AGENTS.md curto na raiz e, se a sua ferramenta não lê, crie o arquivo dela com uma linha: "siga AGENTS.md".

O LOOP E A CHAMADA EVENTUAL

LOOP: próximo turno, tudo de novo, steering incluso

1. VOCÊ PEDE

"revisa esse PR"

2. JANELA DO TURNO, MONTADA DE NOVO A CADA VOLTA

system prompt | STEERING | histórico | arquivos | skill ativa

steering: toda volta, use ou não. **skill ativa:** só depois de chamada.

3. MODELO RESPONDE

e o loop recomeça

FORA DO LOOP: SKILLS INSTALADAS

Só name + description na janela, ~100 tokens cada

- `review-security`
- `pr-summary`
- `pytest-triage`
- `docs-writer`

CHAMADA EVENTUAL

Quando o pedido casa com a descrição, o corpo da skill (até ~5.000 tokens) entra na janela e fica até o fim da sessão. Se nunca casar, nunca paga.

Por isso: steering curto, porque paga em toda volta; skill pode ser longa, porque paga só quando chamada.

STEERING BOM, STEERING RUIM

AGENTS.md (pagamentos-api)

```
# pagamentos-api

- Python 3.10+, stdlib + pytest. Rode: python -m pytest -q
- Dinheiro é int em centavos. Nunca float.
- sqlite3 sem ORM; toda query com parametro ?
- Tokens: secrets.compare_digest, nunca ==
- Datas em UTC, timezone-aware
- PAN e token nunca vao para log
- Webhooks sao idempotentes (event_id unico)
- Antes de entregar: pytest verde
```

BOM

Fatos curtos

cada linha corrige um erro provável; barato e sempre presente

RUIM

Manual disfarçado

40 linhas de passos; ignorado após compactação e custa sempre

Sinal: se tem passos numerados, é procedimento. Procedimento é skill, não steering.

ATIVIDADE 1.1: STEERING DO REPO

Dez minutos para escrever as regras da casa do pagamentos-api

STEERING DO REPO: COMO JOGAR

PASSO 1

Ler

README.md do pagamentos-api: convenções e gotchas.

PASSO 2

Escrever

Até 10 linhas no formato da sua ferramenta (ou AGENTS.md).

PASSO 3

Testar

Sessão nova: "que tipo usamos para dinheiro aqui?"

PASSO 4

Pontuar

+1 por convenção, -1 por procedimento, +2 se o teste passa.

Onde salvar: Kiro em .kiro/steering/conventions.md; Claude Code em CLAUDE.md; Copilot em .github/copilot-instructions.md; neutro em AGENTS.md. Guarde: a skill de hoje vai copiar daqui.

BLOCO 2: SKILL

O método: procedimento empacotado, portável, carregado só quando a tarefa pede

TRÊS JEITOS DE ENSINAR O AGENTE

NA CONVERSA

Prompt

vive	uma sessão
custo	repetir toda vez
falha	cada dev escreve diferente

NO PROJETO

Steering

vive	em todo turno
custo	tokens sempre
falha	vira procedimento gigante

NA PASTA

Skill

vive	quando a tarefa casa
custo	~100 tokens até ativar
ganho	método + scripts portáveis

Regra: fato curto e permanente vai para steering; procedimento vai para skill; ferramenta vai para MCP.

A DEFINIÇÃO OFICIAL

"Pastas organizadas de instruções, scripts e recursos que os agentes descobrem e carregam dinamicamente."

Zhang, Lazuka e Murag (Anthropic), Equipping agents for the real world with Agent Skills, 16/10/2025 (tradução nossa)

PASSO 1

Metadados

name + description de todas as skills, sempre no contexto. ~100 tokens cada.

PASSO 2

Instruções

O corpo do SKILL.md entra quando a tarefa casa com a descrição. < 5.000 tokens.

PASSO 3

Recursos

references/, assets/ lidos sob demanda; scripts/ executados sem entrar no contexto.

TAXONOMIA EM QUATRO EIXOS

MECANISMO	QUANDO CARREGA	QUEM DECIDE	O QUE CARREGA	CONTEXTO
steering sempre	todo turno	ninguém	regras	compartilhado
steering por arquivo	ao tocar arquivo que casa	padrão de arquivo	regras	compartilhado
skill	match de descrição ou /nome	modelo ou humano	método + scripts	compartilhado
plugin / power	palavra-chave da tarefa	modelo	skills + MCP tools	compartilhado
hook	evento: save, tool, stop	determinístico	comando ou prompt	fora do modelo
subagente	delegação	modelo ou nome	prompt + tools próprios	**ISOLADO**
MCP server	sempre registrado	modelo chama a tool	ferramentas	compartilhado

REGRA DE BOLSO: QUAL USAR

STEERING

Padrões do projeto

EXEMPLO

Centavos, sqlite com ?, UTC. Poucas linhas, sempre presentes.

SKILL

Procedimento reutilizável

EXEMPLO

Como revisar um PR; como triar falhas do pytest. Portável entre agentes.

PLUGIN

Ferramenta + método

EXEMPLO

Stripe: MCP tools + skills de checkout, ativadas por palavra-chave.

HOOK

Toda vez, sem falta

EXEMPLO

Ao salvar .py, roda pytest. Não depende do modelo lembrar.

SUBAGENTE

Contexto isolado

EXEMPLO

Três revisores em paralelo, cada um com sua skill e sua janela.

MCP

Ferramentas

EXEMPLO

Ler o Jira, consultar o banco. Dá o que fazer, não como.

UMA PASTA, VÁRIOS AGENTES

code-review/ (padrão agentskills.io)

```
code-review/
├── SKILL.md          # obrigatório
├── scripts/          # executa, não lê
├── references/       # lê sob demanda
├── assets/           # templates
└── evals/            # evals.json
```

Kiro	`.kiro/skills/`
Claude Code	`.claude/skills/`
VS Code + Copilot	`.github/skills/` (+ `.claude/skills/`)
Codex CLI	`.agents/skills/`

Spec aberta: name, description, license, compatibility, metadata, allowed-tools. Campos extras de um cliente são ignorados pelos outros, sem quebrar.

ATIVIDADE 1.2: DISSECAÇÃO

Quatro skills reais na mesa. 12 minutos para abrir, 8 para a revelação

DISSECAÇÃO: COMO JOGAR

1. FRONTMATTER CONTRA A SPEC

name (1 a 64, minúsculas, hífens, igual à pasta) e description (1 a 1024). 1 ponto por violação.

2. ESTIME OS TOKENS DOS 3 NÍVEIS

Metadados, corpo, recursos. 1 ponto por estimativa na ordem certa.

3. PADRÕES E ANTI-PADRÕES

Dois de cada por skill. 2 pontos por padrão nomeado corretamente.

4. RANKING DA MELHOR PARA A PIOR

Uma frase de justificativa. Revelação em 8 min.

4
skills no kit

12
min de dissecação

1
ponto por violação

2
pontos pelo achado escondido

BLOCO 3: ANATOMIA DE UMA SKILL

O que tem dentro e por que cada parte existe

FRONTMATTER: CAMPOS E LIMITES

CAMPO	OBRIG.	LIMITE
name	sim	1 a 64, [a-z0-9-], sem -- e sem hífen nas pontas, igual à pasta
description	sim	1 a 1024 caracteres; o que faz e quando usar
license	não	nome da licença ou arquivo
compatibility	não	até 500; só se exigir ambiente
metadata	não	mapa chave: valor (strings)
allowed-tools	não	experimental; tools pré-aprovadas

SKILL.md

```
---
name: code-review
description: >
  Revisa diffs e PRs em Python
  procurando bugs de correção,
  segurança e desempenho. Use
  quando pedirem review, revisão,
  "olha esse PR" ou antes do merge.
  Não escreve testes.
metadata:
  author: turma-sd
  version: "0.1"
---
```

A DESCRIÇÃO CARREGA O GATILHO

ANTES

Não dispara ou dispara em tudo

texto	"Helps with code."
problema	não diz quando; sem palavras do usuário
efeito	o agente nunca escolhe, ou escolhe sempre

DEPOIS

Dispara no pedido certo

texto	"Revisa diffs e PRs em Python… Use quando pedirem review, revisão, 'olha esse PR' ou antes do merge. Não escreve testes."
por quê	o quê + quando + o que não faz
efeito	casa com o pedido, erra menos no near-miss

Teste mental: a descrição funcionaria se o usuário não dissesse a palavra 'review'? Se não, ela está estreita demais.

PROGRESSIVE DISCLOSURE EM NÚMEROS

~100

tokens por skill instalada (nível 1)

<5.000

tokens do corpo ao ativar (nível 2)

500

linhas: teto recomendado do SKILL.md

0

tokens de um script executado



COMO ESTRUTURAR

Diga quando ler cada arquivo

- "Leia references/cwe.md se encontrar entrada externa"
- não: "veja references/ para detalhes"
- referências a um nível de profundidade
- scripts para o que deve ser determinístico
- gotchas ficam no SKILL.md, não em reference

CAMPOS EXTRAS DAS FERRAMENTAS

CAMPO	EFEITO	ONDE EXISTE
disable-model-invocation: true	só o humano invoca com /nome	Claude Code, Copilot
user-invocable: false	só o modelo invoca; some do menu	Claude Code, Copilot
context: fork	roda a skill em um subagente isolado	Claude Code, Copilot
paths: ["src//*.py"]**	ativa só quando toca arquivos que casam	Claude Code (Kiro: steering fileMatch)
$ARGUMENTS, $0, $1	argumentos do /nome entram no corpo	Claude Code; Copilot via argument-hint
!`git diff HEAD`	roda antes do modelo ler e injeta a saída	Claude Code
hooks:	hooks válidos enquanto a skill está ativa	Claude Code

PADRÕES DE CORPO QUE FUNCIONAM

GOTCHAS

Fatos que o modelo erraria

EXEMPLO

"users usa soft delete: filtre deleted_at IS NULL". Corrigiu o agente? Vira gotcha.

TEMPLATE

Formato de saída concreto

EXEMPLO

A tabela de achados do README. O modelo casa padrão melhor que prosa.

CHECKLIST

Passos com dependência

EXEMPLO

[ ] coletar diff [ ] revisar [ ] validar [ ] só então entregar.

VALIDATION LOOP

Fazer, validar, corrigir

EXEMPLO

Rode validate_report.py; se falhar, corrija e rode de novo.

DEFAULT, NÃO MENU

Uma escolha e uma saída

EXEMPLO

"Use pdfplumber; se for escaneado, pdf2image + pytesseract."

PROCEDIMENTO

Como abordar, não a resposta

EXEMPLO

"1. leia o schema 2. junte por _id 3. filtre", não a query pronta.

ANTI-PADRÕES

VAGO

"Helps with code"

SINTOMA

Nunca dispara, ou dispara em tudo. O agente não tem palavra para casar.

ENCICLOPÉDIA

Explicar o que o modelo já sabe

SINTOMA

"PDF é um formato de arquivo que..." Gasta contexto e enterra a instrução que importa.

MONÓLITO

1.000 linhas, 40 regras

SINTOMA

SEMPRE/NUNCA sem porquê; "em dúvida, reporte" vira fábrica de falso positivo.

GERADO NO VAZIO

Pediu ao LLM sem dar contexto

SINTOMA

"Handle errors appropriately." Procedimento genérico sem gotchas do seu projeto.

CODE REVIEW: O QUE A SKILL CARREGA

CONVENÇÕES DO PROJETO

Já estão no steering da Atividade 1.1. A skill cita as que importam para o review, não repete tudo.

CATEGORIAS E SEVERIDADES

Seis categorias (segurança, correção, desempenho, dinheiro, concorrência, estilo) e quatro severidades.

FORMATO DO ACHADO

arquivo, linha, severidade, categoria, evidência, sugestão. Sem linha e sem trecho não conta.

LISTA DE CWES E O DIFF

references/cwe-top25.md e scripts/collect_diff.sh (diff + mensagem de commit).

YU ET AL. 2024/2026

O que a pesquisa diz

- 7 LLMs, 5 prompts, vs SAST
- LLMs superam ferramentas estáticas
- melhor prompt GPT-4: lista de CWEs
- melhor prompt R1: commit + passo a passo
- acham mais em arquivos com menos tokens

O ACHADO QUE VALE PONTO

reports/grupo-v0.md

```
| arquivo    | linha | severidade | categoria | evidencia                                  | sugestao            |
|------------|-------|------------|-----------|--------------------------------------------|---------------------|
| app/db.py  | 40    | critica    | seguranca | WHERE c.name = '{customer}' via f-string   | parâmetro ?         |
| app/auth.py| 14    | media      | seguranca | token == _expected_token(user)             | compare_digest      |
```



PASSO 1

Gerar

O agente escreve a tabela a partir da sua skill.



PASSO 2

Validar

python scripts/validate_report.py: formato, linha, severidade.



PASSO 3

Pontuar

python scripts/score.py vs gabarito: recall, FP, pontos.

ATIVIDADE 1.3 E DISPUTA 1

Sua primeira skill de code review contra 14 bugs plantados

SKILL V0: PASSO A PASSO



PASSO 1

Criar

Copie o template para a pasta de skills da sua ferramenta e preencha os TODOs.



PASSO 2

Validar

python kit-skills/validate.py na pasta code-review-<grupo>.



PASSO 3

Disparar

Sessão nova: "revisa este repositório". Confirme que a skill carregou.



PASSO 4

Relatar

Salve reports/<grupo>-v0.md e rode validate_report.py até passar.



PASSO 5

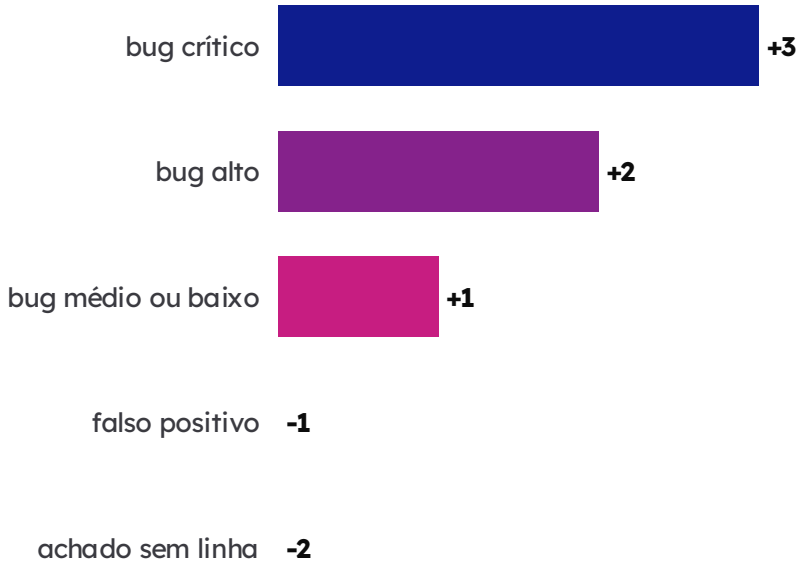
Pontuar

Quando o professor liberar o gabarito: scripts/score.py.



Dica: o steering da 1.1 já diz as regras da casa; a skill diz como revisar. Não repita uma na outra.

DISPUTA 1: CAÇA AOS BUGS



REGRAS

Caça aos bugs, rodada 1

- 14 bugs em 6 categorias no repo
- gabarito lacrado até o fim
- só vale achado no formato
- tolerância de 3 linhas no match
- placar acumula até o Torneio

O QUE FICA E O QUE VEM

FICA

Steering = regras da casa, sempre presentes e curtas.
Skill = método, carregado sob demanda, portável.

NÃO APAGUE NADA

Amanhã você refatora a v0 em uma suíte e compara recall e falsos positivos com hoje.

AMANHÃ (QUA 04/11)

Múltiplas skills: o problema do gatilho, granularidade, subagentes em paralelo.

QUINTA (05/11)

Spec: o que construir agora; as três camadas num projeto; o Torneio Red x Blue.

14

bugs no gabarito

?

o seu recall hoje

72%

meta de amanhã (Rajan)

09/11

entrega da lista


<LAB365>
SENAI